

Fundação CECIERJ - Consórcio CEDERJ
Curso de Tecnologia em Sistemas de Computação
Disciplina: Programação com Interfaces Gráficas (EAD05030)
AP1 1º semestre de 2026

Gabarito

Questão 1 (2,0 pontos) – O código a seguir define características de veículos distintos e uma das ações que eles podem realizar. A classe **Veiculo** é a classe base para as demais e a única ação possível para os veículos é acelerar.

Crie as classes **Carro** e **Moto** que são filhas da classe **Veiculo**, implementando o método **acelerar** de forma que a moto aumente sua velocidade em 5 km/h e o carro aumente em 2 km/h. Além disso, ao alcançar a velocidade máxima, não deve ser possível acelerar mais, ou seja, ao tentar acelerar, a velocidade máxima será mantida.

A inicialização das classes criadas deve ser feita utilizando o construtor da classe pai. O valor do atributo **velocidade_maxima** é de livre escolha e o atributo **num_rodas** deve estar de acordo com a classe filha inicializada.

RESPOSTA

```
class Veiculo:
    def __init__(self, velocidade_maxima, num_rodas):
        self._velocidade_maxima = velocidade_maxima
        self._num_rodas = num_rodas
        self._velocidade_atual = 0

    def acelerar(self):
        pass

class Carro(Veiculo):
    def __init__(self):
        super().__init__(velocidade_maxima=200, num_rodas=4)

    def acelerar(self):
        self._velocidade_atual += 2
        if self._velocidade_atual > self._velocidade_maxima:
            self._velocidade_atual = self._velocidade_maxima

class Moto(Veiculo):
    def __init__(self):
        super().__init__(velocidade_maxima=180, num_rodas=2)

    def acelerar(self):
        self._velocidade_atual += 5
        if self._velocidade_atual > self._velocidade_maxima:
            self._velocidade_atual = self._velocidade_maxima

if __name__ == "__main__":
    carro = Carro()
    moto = Moto()

    veiculos = [carro, moto]
```

```
for veiculo in veiculos:
    veiculo.acelerar()
```

Questão 2 (2,0 pontos) – O código a seguir possui um erro. Qual é o erro e o que ocorre quando o código é executado? Corrija o erro.

```
1 class c:
2     def f(self, a, b):
3         return a / b
4
5 a = c()
6
7 r = a.f(10, 0)
8 print(r)
```

RESPOSTA

O código não trata divisão por zero, de forma que ao passar o valor de $b = 0$, o código gera o erro `ZeroDivisionError` e finaliza. O erro pode ser corrigido utilizando a cláusula `try/except`, da seguinte forma:

```
class c:
    def f(self, a, b):
        try:
            return a / b
        except ZeroDivisionError:
            print("Divisão por zero, tente novamente")
            return "infinito"

a = c()

r = a.c(10, 0)
print(r)
```

Questão 3 (3,0 pontos) – O código a seguir implementa uma representação de uma conta bancária simples.

- a) O que é impresso com a execução do código?
- b) Existe um erro na implementação da classe. Identifique-o e corrija-o.

```
1 class ContaBancaria:
2     def __init__(self, titular, saldo_inicial):
3         self.__titular = titular
4         self.__saldo = saldo_inicial
5
6     def depositar(self, valor):
7         self.__saldo += valor
8
9     def sacar(self, valor):
10        if valor <= self.__saldo:
```

```

11         self.__saldo -= valor
12     else:
13         print("Saldo insuficiente.")
14
15     def get_saldo(self):
16         return self.__saldo
17
18     def set_titular(self, novo_titular):
19         self.__titular = novo_titular
20
21 conta = ContaBancaria("João", 1000)
22 print("Saldo inicial:", conta.get_saldo())
23
24 conta.depositar(500)
25 print("Saldo após depósito:", conta.get_saldo())
26
27 conta.sacar(200)
28 print("Saldo após saque:", conta.get_saldo())
29
30 conta.set_titular("Maria")
31 print("Novo titular da conta:", conta.__titular)
32

```

RESPOSTA

O código imprime:

```

Saldo inicial: 1000
Saldo após depósito: 1500
Saldo após saque: 1300

```

AttributeError: 'ContaBancaria' object has no attribute '__titular'

Correção:

-- opção ruim: tornar o atributo titular público ou protegido

```

self._titular = titular
ou
self.titular = titular

```

-- opção boa: criar um getter para o titular

```

def get_titular(self):
    return self.__titular

```

Questão 4 (3,0 pontos) – O trecho de código abaixo atualiza parte do estado persistente de um jogo ao final de cada partida:

```

1     def atualizar_estado(estado, turnos_sobrevividos, bombas_usadas,
2         ↪ obstaculos_destruidos, obstaculos_totais):

```

```

3     partidas_anteriores = estado["total_partidas"]
4     media_turnos_anterior = estado["media_turnos"]
5     media_bombas_anterior = estado["media_bombas"]
6
7     novo_total_partidas = partidas_anteriores + 1
8
9     nova_media_turnos = (media_turnos_anterior * partidas_anteriores +
10     ↪ turnos_sobrevividos) / novo_total_partidas
11
12     nova_media_bombas = (media_bombas_anterior * partidas_anteriores +
13     ↪ bombas_usadas) / novo_total_partidas
14
15     taxa_destruicao = obstaculos_destruidos / obstaculos_totais
16
17     estado["total_partidas"] = novo_total_partidas
18     estado["media_turnos"] = nova_media_turnos
19     estado["media_bombas"] = nova_media_bombas
20     estado["taxa_destruicao"] = taxa_destruicao
21
22     return estado
23
24 def calcular_inimigos(estado):
25     return 2 + (estado["media_turnos"] // 20)

```

Antes da execução da função `atualizar_estado`, o estado persistente era:

```

estado = {
    "total_partidas": 4,
    "media_turnos": 30.0,
    "media_bombas": 3.0,
    "taxa_destruicao": 0.25
}

```

Na partida recém-encerrada, o jogador obteve:

```

turnos_sobrevividos = 50
bombas_usadas = 5
obstaculos_destruidos = 18
obstaculos_totais = 40

```

Após executar a função `atualizar_estado` com as informações de estado da última partida, responda o que se pede:

- (1,0) a. Qual será o valor final a ser armazenado no estado persistente?
- (1,0) b. Qual será o valor retornado por `calcular_inimigos(estado)` após a atualização de estado?
- (1,0) c. Suponha que o jogo continue e o jogador sempre sobreviva 50 turnos em todas as partidas futuras. A `media_turnos` crescerá indefinidamente? Justifique sua resposta.

RESPOSTA A

```
estado = {
    "total_partidas": 5,
    "media_turnos": 34.0,
    "media_bombas": 3.4,
    "taxa_destruicao": 0.45
}
```

RESPOSTA B

A função retorna "3".

RESPOSTA C

Não. A média é calculada como $\text{nova_media} = (\text{media_anterior} * n + 50) / (n + 1)$. Se a `media_anterior` for menor que 50, a `nova_media` aumenta. Se for maior que 50, a `nova_media` diminui. Como cada `nova_media` é uma combinação ponderada entre a média anterior e 50, quanto maior o número de partidas, menor o impacto de cada novo valor isolado. Assim, à medida que a quantidade de partidas aumenta, a média converge para 50, sendo que o crescimento desacelera progressivamente.

Questão 1 (2,0 pontos) – O que é impresso pelo trecho de código abaixo?

```
class C:
    __l = 1
    __h = 2
    def __init__(self,r=[]):
        self.r = r
    def f(self, l, h):
        self.r = [l,h]
        return C(self.r)
    def __repr__(self):
        return (str(self.r))
    def g(self, l, h):
        self.r = [l,h]
        return self.r
    def __add__(self,x):
        return(self.r[0]*self.r[1]+x.r[0]*x.r[1])
    def __eq__(self,x):
        return((self.r[0]==x.r[0]) and (self.r[1]==x.r[1]))

a = C()
print(a)
try:
    print(a.__h)
except Exception as e:
    print(e,"Error")
else:
    print("No Error")
finally:
    print("xxx")
b = a.f(1,2)
print(a==b)
a.g(2,1)
print(a)
print(b)
print(a+b)
```

RESPOSTA

```
[]  
'C' object has no attribute '__h' Error  
xxx  
True  
[2, 1]  
[1, 2]  
4
```

Questão 2 (3,0 pontos) – O código a seguir define características de veículos distintos e duas das ações que eles podem realizar. A classe **Veiculo** é a classe base para as demais e a única ação possível para os veículos é acelerar.

Crie as classes **Carro** e **Moto** que são filhas da classe **Veiculo**, implementando os métodos **acelerar** e **frear**. O método **acelerar** deve aumentar a velocidade da moto em 5 km/h e do carro em 2 km/h. Deve haver um limite de velocidade máxima para cada veículo.

A inicialização das classes criadas deve ser feita utilizando o construtor da classe pai. O valor do atributo **velocidade_maxima** é de livre escolha e o atributo **num_rodas** deve estar de acordo com a classe filha inicializada.

RESPOSTA

```
class Veiculo:  
    def __init__(self, velocidade_maxima, num_rodas):  
        self._velocidade_maxima = velocidade_maxima  
        self._num_rodas = num_rodas  
        self._velocidade_atual = 0  
  
    def acelerar(self):  
        pass  
  
    def frear(self):  
        pass  
class Carro(Veiculo):  
    def __init__(self):  
        super().__init__(velocidade_maxima=200, num_rodas=4)  
  
    def acelerar(self):  
        if self._velocidade_atual < self._velocidade_maxima - 2:  
            self._velocidade_atual += 2  
        else:  
            self._velocidade_atual = self._velocidade_maxima  
  
    def frear (self):  
        if self._velocidade_atual >= 3:  
            self._velocidade_atual -= 3  
        else:  
            self._velocidade_atual = 0  
  
class Moto(Veiculo):  
    def __init__(self):  
        super().__init__(velocidade_maxima=180, num_rodas=2)
```

```

def acelerar(self):
    if self._velocidade_atual < self._velocidade_maxima - 5:
        self._velocidade_atual += 5
    else:
        self._velocidade_atual = self._velocidade_maxima

def frear (self):
    if self._velocidade_atual >= 6:
        self._velocidade_atual -= 6
    else:
        self._velocidade_atual = 0

if __name__ == "__main__":
    carro = Carro()
    moto = Moto()

    veiculos = [carro, moto]

    for veiculo in veiculos:
        veiculo.acelerar()

    for veiculo in veiculos:
        veiculo.frear()
  
```

Questão 3 (4,0 pontos) – Escreva uma classe `ListaSemRepeticao` que cria listas numéricas ordenadas em ordem crescente. Também deve ser possível combinar dois objetos `ListaSemRepeticao` ou um objeto `ListaSemRepeticao` e uma lista numérica, resultando em uma lista ordenada sem repetição. Por fim, a classe deve prover métodos que retornem o número total de elementos repetidos removidos e a quantidade individual de elementos. A impressão do objeto `ListaSemRepeticao` deve ser igual à de uma lista comum, isto é, na forma `[a, b, c, d]`.

Exemplo:

```

## L1 = [2, 1, 1, 3, 3, 4]
## L1 sem repetição = [1, 2, 3, 4],
## Total de elementos repetidos removidos = 2,
## Qtd. indiv. de elementos repetidos removidos
##     = {1: 1, 2: 0, 3: 1, 4: 0}
##
## L2 = [1, 2, 4, 4]
## L2 sem repetição = [1, 2, 4],
## Total de elementos repetidos removidos = 1,
## Qtd. individual de elementos repetidos removidos
##     = {1: 0, 2: 0, 4: 1}
##
## L1 sem repetição e L2 sem repetição unificadas = [1, 2, 3, 4],
## Total de elementos repetidos removidos = 3,
## Qtd. individual de elementos repetidos removidos
##     = {1: 1, 2: 1, 3: 0, 4: 1}
##
## L1 sem repetição e L2 com repetição unificadas [1, 2, 3, 4]
## Total de elementos repetidos removidos = 4
  
```

```
## Qtd. individual de elementos repetidos removidos
##      = {1: 1, 2: 1, 3: 0, 4: 2}
```

Estrutura mínima da classe:

```
class ListaSemRepeticao:
    def __init__(...):
        ...

    def GetTotalRep(...): # para retornar o número total
        ...             # de elementos repetidos removidos

    def GetIndividualRep(...): # para retornar o número
        ...                   # individual de elementos
        ...                   # repetidos removidos

    def Merge(...): # para unificar listas
        ...

    def __repr__(...): # para imprimir corretamente
        ...
```

RESPOSTA

```
class ListaSemRepeticao:
    def __init__(self, elementos=None):
        if elementos is None:
            elementos = []
        # Conta repetições
        self._repetidos = {}
        for e in elementos:
            self._repetidos[e] = self._repetidos.get(e, 0) + 1

        # Remove duplicatas e ordena
        self._lista = sorted(set(elementos))

    def GetTotalRep(self):
        """Retorna o número total de elementos repetidos removidos"""
        return sum(v - 1 for v in self._repetidos.values())

    def GetIndividualRep(self):
        """Retorna a quantidade de repetições de cada elemento"""
        return {k: v - 1 for k, v in self._repetidos.items() if v > 1}

    def Merge(self, outra):
        """Unifica com outra ListaSemRepeticao ou lista comum"""
        if isinstance(outra, ListaSemRepeticao):
            nova_lista = self._lista + outra._lista
        elif isinstance(outra, list):
            nova_lista = self._lista + outra
        else:
            raise TypeError("Merge aceita apenas ListaSemRepeticao ou list")

        return ListaSemRepeticao(nova_lista)

    def __repr__(self):
        return repr(self._lista)
```


Questão 4 (1,0 ponto) – Explique o que o seguinte trecho de programa faz.

```
import sys

def main():
    try:
        facaAlgo()
        return 0
    except:
        return 1

if __name__ == "__main__":
    sys.exit(main())
```

RESPOSTA

A primeira parte define a função `main()`:

Dentro dela, tenta executar a função `facaAlgo()`.

Se `facaAlgo()` for executada sem erro, retorna 0.

Se ocorrer qualquer exceção, retorna 1.

A segunda parte é o bloco principal do programa:

O programa verifica se está sendo executado diretamente (`if __name__ == "__main__":`).

Nesse caso, chama `main()` e encerra o programa com `sys.exit()`, retornando o valor de `main()` como código de saída do sistema.